```
  "keywords": [
    "node",
    "glitch",
    "express"
  ]
}
```

Add the packages given below to the dependencies section, add Cheerio and Request to manage our scraping, and *express-http-to-https* to manage redirection of *http:// requests to https://*.

```
    "cheerio": "^0.22.0",
    "request": "^2.88.2",
    "express-http-to-https": "^1.1.4"
```

Now we need to decide the source for our data to be shown. Ideally, this should have been a Web API that we can invoke to get the coronavirus data dynamically. Unfortunately, no such Web API seems to exist and creating one will be a project on its own. So, we must get by with existing Web pages that are doing a reasonably good job of keeping the overall coronavirus statistics up to date with latest inputs coming from all countries. Three such sites I could find were *nCoV2019.live* by Avi Schiffmann, the COVID-19 map by CNA, and the Coronavirus Update by *Worldometers.info*. After checking the information and the way it is presented, the Worldometers site seems ideal, as the information about the number of cases and deaths is updated in the title of the Web page itself, reducing the complexity of the scraping process.

Server.js and startServer

First, we express in Server.js that we need the Cheerio and Request libraries also to be supported, as shown in the code below. We then add dependencies to the *express-http-to-https* library.

```
// we've started you off with Express
(https://expressjs.com/)
// but feel free to use whatever
libraries or frameworks you'd like
through `package.json`.
const express = require("express");
const cheerio = require("cheerio");
const request = require("request");
const redirectToHTTPS =
require('express-http-to-https').
redirectToHTTPS;
```

The server processing starts with a *startServer();* call. The sections below explain the non-trivial code we added, namely, to instantiate the *express()* instance, and make the HTTP to HTTPS redirection a part of PWA solutions that work only with HTTPS requests. There is more about this in the latter part of the article. The most important thing is to register our function *getCounts* as the method which will serve data when we get a *Get* request for */counts* or */counts/*. There is some more boilerplate code for standard logging and serving public files, which we will not go into right now. The core of our processing is in the *getCounts* function, which we will see in the next section.

Make an Express app instance. Register for HTTP to HTTPS redirection. This can be ignored for now but is needed for PWA purposes.

Figure 2: Show *Next to the Code*

```
const app = express(); // express app
instance
  // Redirect HTTP to HTTPS,
  app.use(redirectToHTTPS([/
localhost:(\d{4})/], [], 301));
```

Type the standard logging code, followed by code that registers */counts* or */counts/* to *getCounts()* function.

```
// Logging for each request
  app.use((req, resp, next) => {
    const now = new Date();
    const time = `${now.
toLocaleDateString()} - ${now.
toLocaleTimeString()}`;
    const path = `"${req.method} ${req.
path}"`;
    const m = `${req.ip} - ${time} -
${path}`;

    // eslint-disable-next-line no-
console
    console.log(m);
    next();
  });

  // Handle requests for the data
  app.get("/counts/", getCounts);
  app.get("/counts", getCounts);
```

Make public files available and send back *index.html* as response. Then log the listener port.

```
  // make all the files in 'public'
available
  // https://expressjs.com/en/starter/
static-files.html
  app.use(express.static("public"));
  // https://expressjs.com/en/starter/
basic-routing.html
  app.get("/", (request, response) => {
    response.sendFile(__dirname + "/
views/index.html");
  });
  // listen for requests :)
  const listener = app.listen(process.
env.PORT, () => {
    console.log("Your app is listening
on port " + listener.address().port);
  });
```